

PC 601 AI — WEB PROGRAMMING

Question Bank: 2 Marks & 6 Marks Questions with Answers

PC 601 AI — WEB PROGRAMMING

Question Bank (2 Marks & 6 Marks Questions with Answers)

B.E. AI/ML — Osmania University Prepared with reference to the prescribed textbooks: Robert W. Sebesta, “Programming the World Wide Web” (3rd Ed., Pearson, 2006); Wendy Willard, “HTML5” (McGraw Hill, 2013); Thomas Powell, “Ajax: The Complete Reference” (Tata McGraw-Hill, 2011); and the course syllabus.

UNIT – I: WWW Basics & HTML5

2 Marks Questions

Q1. What is the World Wide Web (WWW)? The World Wide Web is a system of interlinked hypertext documents (web pages) accessed over the Internet, identified by URLs, and retrieved using protocols such as HTTP, allowing users to navigate between related resources via hyperlinks using a web browser.

Q2. What is a Web Browser? Name any two. A web browser is a client application that requests web resources (HTML pages, images, scripts, etc.) from web servers, renders/displays them to the user, and handles user interaction (clicks, form submissions). Examples include Google Chrome and Mozilla Firefox.

Q3. What is a Web Server? A web server is a program (running on a host machine) that listens for incoming HTTP requests from clients (browsers), retrieves or generates the requested resource (a static file or dynamically generated content), and sends it back to the client as an HTTP response.

Q4. What is a URL, and what are its main components? A URL (Uniform Resource Locator) is a string that specifies the location and means of accessing a resource on the web. Its main components are: **scheme** (e.g., https), **host** (domain name/IP), **port** (optional, e.g., :8080), **path** (resource location on the server), and optionally a **query string** (?key=value) and a **fragment** (#section).

Q5. What is HTTP? HTTP (HyperText Transfer Protocol) is the application-layer protocol used for communication between web browsers and web servers, based on a request-response model, where a client sends a request (with a method like GET or POST) and the server returns a response (status code + headers + body).

Q6. Name any four common HTTP methods. GET (retrieve a resource), POST (submit data to be processed), PUT (update/replace a resource), and DELETE (remove a resource) are four common HTTP methods.

Q7. What is HTML5? HTML5 is the fifth major revision of the HyperText Markup Language, introducing new semantic elements (e.g., <header>, <article>, <section>), native multimedia support (<audio>, <video>), new form input types, and APIs (e.g., Canvas, Geolocation, Local Storage) without requiring third-party plugins like Flash.

Q8. What is a Hyperlink? Give the HTML tag used to create one. A hyperlink is a reference/link that allows navigation from one web resource to another when clicked. It is created using the <a> (anchor) tag with an href attribute, e.g., Next Page.

Q9. Which HTML5 tag is used to embed images, and what is the purpose of the alt attribute? The tag is used to embed images, e.g., . The alt attribute provides alternative text describing the image, displayed if the image fails to load, and used by screen readers for accessibility.

Q10. Name the HTML5 tags used for embedding audio and video without plugins. <audio> and <video> are the HTML5 tags used to natively embed audio and video content, respectively, without requiring external plugins like Flash.

Q11. Differentiate between an ordered list and an unordered list in HTML. An ordered list () displays items with sequential numbering/lettering, while an unordered list () displays items with bullet points; in both cases, individual items are defined using the tag.

Q12. What HTML tags are used to create a table? <table> defines the table; <tr> defines a table row; <th> defines a header cell; and <td> defines a standard data cell.

Q13. What is the purpose of the <form> tag in HTML? The <form> tag defines an interactive section of a web page used to collect user input (via elements like <input>, <select>, <textarea>) and submit it to a server, specified by the form's action (target URL) and method (e.g., GET or POST) attributes.

Q14. How can forms be styled in HTML5? Forms can be styled using CSS, either by applying styles to form elements via class/id selectors, or by using CSS pseudo-classes specific to form states (e.g., :focus, :invalid, :required) to visually indicate validation status or interaction state.

6 Marks Questions

Q1. Explain the working of the World Wide Web, describing the roles of Web Browsers, Web Servers, URLs, and HTTP together with an example of a complete request-response cycle.

The World Wide Web operates on a **client-server** model built on several cooperating components:

1. **URL (Uniform Resource Locator):** Every resource on the web (a page, image, script) is identified by a URL, e.g., `https://www.example.com:443/products/item.html?id=5#reviews`. This URL specifies the **scheme** (`https`, indicating HTTP over TLS), the **host** (`www.example.com`, resolved via DNS to an IP address), the **port** (`443`, the default for HTTPS), the **path** (`/products/item.html`, identifying the specific resource on the server), a **query string** (`?id=5`, passing parameters to the server), and a **fragment** (`#reviews`, identifying a specific section within the retrieved page, handled entirely client-side).
2. **Web Browser (Client):** When a user enters or clicks a URL, the browser parses it, uses DNS to resolve the hostname to an IP address, and establishes a TCP connection (with a TLS handshake, if HTTPS) to the web server at that address and port.
3. **HTTP Request:** The browser sends an **HTTP request** over this connection, specifying a method (typically GET for retrieving a page), the requested path, protocol version, and various headers (e.g., Host, User-Agent, Accept, Cookie).
4. **Web Server:** The web server (e.g., Apache, Nginx) receives this request, and either serves a static file directly from disk (for simple resources) or forwards the request to an application/server-side script (e.g., a PHP script or Java Servlet) to dynamically generate the response content.
5. **HTTP Response:** The server sends back an **HTTP response**, consisting of a status line (e.g., HTTP/1.1 200 OK), response headers (e.g., Content-Type: text/html, Content-Length), and the response body (the actual HTML/data content).
6. **Rendering:** The browser receives this response and **renders** the HTML content, applying any linked CSS and executing any linked JavaScript, and if the HTML references further resources (images, stylesheets, scripts), the browser issues additional HTTP requests to fetch each of those as well, repeating this entire cycle for each embedded resource until the full page is assembled and displayed.

This request-response cycle, repeated as needed for every resource referenced by a page, is the fundamental mechanism underlying all web browsing, and understanding it is essential background for the more advanced topics (Ajax, servlets, PHP) covered later in the course, all of which build upon this basic HTTP client-server model.

Q2. Explain HTML5 and describe its key new features compared to earlier HTML versions, with examples.

HTML5 is the modern standard version of HTML, designed to natively support rich, interactive, multimedia-capable web applications without relying on third-party browser plugins (like Adobe Flash, which HTML5 effectively made obsolete). Its key new features include:

1. **Semantic Elements:** HTML5 introduced meaningful structural tags such as `<header>`, `<nav>`, `<article>`, `<section>`, `<aside>`, and `<footer>`, replacing the

earlier common practice of using generic `<div>` elements with descriptive `class/id` names for the same purpose. This improves code readability, and importantly, helps search engines and assistive technologies (screen readers) better understand a page's structure.

```
<header><h1>My Blog</h1></header>
<nav><a href="/">Home</a> | <a href="/about">About</a></nav>
<article>
  <h2>My First Post</h2>
  <p>Content of the post goes here...</p>
</article>
<footer>© 2026 My Blog</footer>
```

2. **Native Multimedia Support:** The `<audio>` and `<video>` tags allow embedding audio/video content directly, with built-in browser playback controls, eliminating the earlier dependency on Flash or other plugins:

```
<video width="640" controls>
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support the video tag.
</video>
```

3. **New Form Input Types and Validation:** HTML5 added new `<input>` types (e.g., email, date, number, range, color, url) that trigger appropriate mobile keyboards/UI controls and enable built-in **client-side validation** without JavaScript (e.g., using the `required`, `pattern`, `min`, `max` attributes), reducing the need for custom validation scripts for common cases.
4. **New APIs:** HTML5 introduced several JavaScript APIs directly into the browser platform, including the **Canvas API** (for 2D drawing/graphics), **Geolocation API** (to access the user's location, with permission), **Web Storage** (`localStorage/sessionStorage`, for storing data client-side without cookies), and **Web Workers** (for running JavaScript in background threads).
5. **Improved Structure and Doctype:** HTML5 simplified the document type declaration to simply `<!DOCTYPE html>` (compared to the much longer, more complex DTD declarations required in HTML4/XHTML).

Together, these features let HTML5 support rich, app-like web experiences natively in the browser, which is why HTML5 (along with JavaScript and CSS3) forms the foundation of essentially all modern web development, including the later syllabus topics of dynamic client-side scripting (JavaScript/Ajax) and server-side application frameworks.

Q3. Explain how to create HTML Forms, including common input elements, and describe how form data is submitted to a server.

The `<form>` element defines a section of a page that collects user input and submits it to a server for processing. Its key attributes are `action` (the URL the data is sent to) and `method` (typically GET or POST).

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
```

```

<input type="text" id="name" name="username" required><br>

<label for="email">Email:</label>
<input type="email" id="email" name="email" required><br>

<label for="age">Age:</label>
<input type="number" id="age" name="age" min="1" max="120"><br>

<label for="gender">Gender:</label>
<select id="gender" name="gender">
  <option value="male">Male</option>
  <option value="female">Female</option>
</select><br>

<input type="checkbox" name="subscribe" value="yes"> Subscribe to
newsletter<br>

<input type="radio" name="plan" value="basic"> Basic
<input type="radio" name="plan" value="premium"> Premium<br>

<textarea name="comments" rows="4" cols="30">Enter comments
here</textarea><br>

<button type="submit">Submit</button>
</form>

```

Common input elements: <input> (with various type values: text, password, email, checkbox, radio, number, date, file, etc.), <select>/<option> (dropdown lists), <textarea> (multi-line text input), and <button> (submit/reset/generic buttons).

Submission process: When the user clicks the submit button, the browser collects the current value of every named form element and encodes it, then sends it to the server according to the specified method: - **GET:** All form data is appended to the action URL as a query string (e.g., /submit?username=John&email=john@x.com), visible in the URL, suitable for simple, non-sensitive, idempotent requests (like search forms), and subject to URL length limits. -

POST: Form data is sent in the **body** of the HTTP request (not appended to the URL), suitable for larger amounts of data, sensitive data (e.g., passwords — though POST alone does not encrypt data; HTTPS is needed for that), and operations that change server state (e.g., creating an account).

The server-side application (a PHP script, Java Servlet, etc., as covered in later units) then reads and processes this submitted data to generate an appropriate response.

Q4. Explain the styling of HTML forms using CSS, and discuss CSS pseudo-classes relevant to form validation states.

Styling forms with CSS improves usability and provides visual feedback to users. Common styling techniques include:

```

<style>
  input[type="text"], input[type="email"] {
    width: 250px;
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    margin-bottom: 10px;
  }
  input:focus {
    border-color: #4CAF50;
    outline: none;
    box-shadow: 0 0 4px #4CAF50;
  }
  input:required:invalid {
    border-color: red;
  }
  input:required:valid {
    border-color: green;
  }
  button[type="submit"] {
    background-color: #4CAF50;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
  }
  button[type="submit"]:hover {
    background-color: #45a049;
  }
</style>

```

Key CSS selectors/pseudo-classes for forms:

- **: focus** — applies styling when an input element currently has keyboard focus (e.g., highlighting the active field for clarity).
- **: required** — targets input elements that have the required attribute set.
- **: valid / : invalid** — apply styling based on whether the current input value satisfies the field's validation constraints (e.g., a valid email format for type="email", or a number within min/max bounds) — this is HTML5's built-in **client-side constraint validation**, and CSS can visually respond to it without any JavaScript.
- **: checked** — targets checkboxes/radio buttons that are currently selected, useful for custom-styling checkbox/radio appearances.
- **: disabled** — applies styling to disabled form elements (e.g., a greyed-out appearance).

By combining these selectors with HTML5's built-in validation attributes (required, pattern, min, max, type="email", etc.), a form can provide real-time, in-browser visual feedback (e.g., turning a field's border green when valid, red when invalid) without writing any custom JavaScript validation logic, which significantly improves the user experience for simple validation needs, though more complex validation logic (covered in Unit III) typically still requires JavaScript.

UNIT – II: XML

2 Marks Questions

Q1. What is XML? XML (eXtensible Markup Language) is a markup language designed to store, structure, and transport data in a human-readable, self-descriptive, platform-independent format, using custom-defined tags rather than a fixed set of tags (unlike HTML, which has a predefined tag vocabulary for display purposes).

Q2. What are the basic rules for a well-formed XML document? A well-formed XML document must: have exactly one root element; every opening tag must have a matching closing tag (or be a self-closing tag); tags must be properly nested (no overlapping); attribute values must be quoted; and tag names are case-sensitive.

Q3. What is a DTD (Document Type Definition)? A DTD defines the legal structure of an XML document — the allowed elements, their attributes, and the order/nesting rules for child elements — allowing an XML parser to validate whether a given document conforms to the expected structure.

Q4. What is an XML Namespace, and why is it used? An XML Namespace is a mechanism to avoid element/attribute name conflicts when combining XML vocabularies from different sources within a single document, achieved by qualifying element names with a unique URI-based prefix (e.g., `<h:table>` vs `<f:table>`, where `h` and `f` are declared namespace prefixes referring to different URIs).

Q5. What is an XML Schema (XSD)? How does it differ from a DTD? An XML Schema (XSD) is a more powerful and expressive alternative to a DTD for defining and validating the structure of an XML document, itself written in XML syntax, and supporting richer data typing (e.g., integers, dates), namespaces, and more precise constraints than a DTD offers.

Q6. How can XML documents be displayed/styled using CSS? An XML document can be linked to a CSS stylesheet using a processing instruction (`<?xml -stylesheet type="text/css" href="style.css"?>`), which a browser can use to apply visual styling (fonts, colors, layout) directly to the XML elements for display, similar to how CSS styles HTML.

Q7. What is XPath? XPath (XML Path Language) is a query language used to navigate and select specific nodes (elements, attributes, text) within an XML document's tree structure, using path expressions similar to file-system paths (e.g., `/bookstore/book[1]/title`).

Q8. What is XSLT? XSLT (eXtensible Stylesheet Language Transformations) is a language used to transform an XML document into another format (e.g., HTML, plain text, or a different XML structure), by defining template rules that match specific nodes (often using XPath expressions) and specify the corresponding output.

Q9. What is an XML Processor (Parser)? An XML processor (parser) is software that reads an XML document, checks it for well-formedness (and optionally validity against a DTD/Schema), and provides programmatic access to its structure/content, typically via one of two models: **DOM** (builds an in-memory tree of the whole document) or **SAX** (event-based, streaming through the document sequentially without loading it entirely into memory).

Q10. Differentiate between DOM and SAX XML parsing. DOM (Document Object Model) parsing loads the entire XML document into memory as a tree structure, allowing random access and modification, but consuming more memory for large documents. SAX (Simple API for XML) parsing is event-driven and streams through the document sequentially, triggering callback events as elements are encountered, using much less memory but only allowing forward, read-only traversal.

6 Marks Questions

Q1. Explain the structure of a well-formed XML document with an example, and describe the syntax rules that must be followed.

An XML document consists of a **prolog** (optional XML declaration specifying version/encoding), followed by exactly **one root element**, which can contain nested child elements, attributes, and text content, forming a hierarchical tree structure.

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="fiction">
    <title lang="en">The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <year>1925</year>
    <price>10.99</price>
  </book>
  <book category="non-fiction">
    <title lang="en">Sapiens</title>
    <author>Yuval Noah Harari</author>
    <year>2011</year>
    <price>15.50</price>
  </book>
</bookstore>
```

Syntax rules for a well-formed XML document:

- 1. Single root element:** The entire document must be contained within one, and only one, top-level element (here, <bookstore>).
- 2. Properly closed tags:** Every opening tag must have a corresponding closing tag (e.g., <title>...</title>), or be self-closing for empty elements (e.g.,
).
- 3. Proper nesting:** Tags must be correctly nested without overlapping — e.g., <a> is invalid; it must be <a>.
- 4. Case sensitivity:** XML tags are case-sensitive; <Book> and <book> are treated as entirely different elements.
- 5. Quoted attribute values:** Attribute values must always be enclosed in quotes (e.g., category="fiction", not category=fiction).
- 6. No standalone special characters in content:** Special characters like <, >, and & appearing in actual text content must be escaped using entity references (<, >, &) since they have special meaning in XML syntax.

An XML document satisfying all these rules is considered **well-formed**; it may additionally be checked for **validity** against a DTD or XML Schema, which imposes further constraints on which specific elements/attributes are allowed and in what structure/order (validity is a stronger, optional check beyond basic well-formedness).

Q2. Explain Document Type Definition (DTD) with an example, and describe how it is used to validate an XML document.

A DTD specifies the set of legal elements and attributes for an XML document, and the rules governing their structure (e.g., which child elements an element may contain, and in what order/quantity).

Example DTD (inline, for the bookstore XML above):

```
<!DOCTYPE bookstore [  
  <!ELEMENT bookstore (book+)>  
  <!ELEMENT book (title, author, year, price)>  
  <!ATTLIST book category CDATA #REQUIRED>  
  <!ELEMENT title (#PCDATA)>  
  <!ATTLIST title lang CDATA #IMPLIED>  
  <!ELEMENT author (#PCDATA)>  
  <!ELEMENT year (#PCDATA)>  
  <!ELEMENT price (#PCDATA)>  

```

Explanation of the DTD rules: - `<!ELEMENT bookstore (book+)>` declares that a bookstore element must contain **one or more** (+) book child elements. - `<!ELEMENT book (title, author, year, price)>` declares that each book must contain exactly these four child elements, **in this specific order**. - `<!ATTLIST book category CDATA #REQUIRED>` declares that book must have a category attribute (of type character data), and it is **required** (must be present). - `<!ELEMENT title (#PCDATA)>` declares that title contains only parsed character data (plain text, no further nested elements). - `<!ATTLIST title lang CDATA #IMPLIED>` declares an optional (#IMPLIED) lang attribute on title.

Validation process: An XML parser that supports DTD validation reads both the DTD (either declared inline as above, or referenced externally via `<!DOCTYPE bookstore SYSTEM "bookstore.dtd">`) and the XML document itself, and checks that every element/attribute in the document conforms to the rules specified in the DTD — e.g., it would flag an error if a book element were missing its required category attribute, or if its child elements appeared in the wrong order, or if an entirely undeclared element appeared somewhere in the document. This validation step is what elevates a document from merely “well-formed” (correct XML syntax) to “valid” (also conforming to an agreed-upon structural contract), which is essential when XML documents are exchanged between different systems/organizations that need to agree on a common data structure.

Q3. Explain XML Namespaces and XML Schema, and discuss why XML Schema is generally preferred over DTDs in modern applications.

XML Namespaces: When combining XML content from multiple different vocabularies within a single document (e.g., embedding a specific “furniture catalog” schema’s `<table>` element alongside an HTML-like `<table>` element for layout), naming **collisions** can occur, since both use the same tag name for entirely different meanings. XML Namespaces solve this by associating each vocabulary with a unique URI (which serves only as a unique identifier, not necessarily a real, dereferenceable web address), and requiring elements from that vocabulary to be prefixed accordingly:

```

<root xmlns:h="http://www.example.com/html-tables"
      xmlns:f="http://www.example.com/furniture">
  <h:table>
    <h:tr><h:td>Cell content</h:td></h:tr>
  </h:table>
  <f:table>
    <f:name>Coffee Table</f:name>
    <f:width>120cm</f:width>
  </f:table>
</root>

```

Here, `h:table` and `f:table` are unambiguously distinguished as belonging to different namespaces (declared via the `xmlns:h` and `xmlns:f` attributes), even though both use the local name “table.”

XML Schema (XSD): An XML Schema is a more powerful, modern alternative to a DTD for both documenting and validating an XML document’s structure, itself expressed using XML syntax (unlike DTD’s own distinct, non-XML syntax):

```

<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="year" type="xs:integer"/>
      <xs:element name="price" type="xs:decimal"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```

Why XML Schema is generally preferred over DTD:

- 1. Written in XML itself:** Unlike a DTD (which uses its own distinct, non-XML syntax), an XSD is itself a valid XML document, so it can be parsed, processed, and even validated using the same standard XML tools used for any other XML document.
- 2. Rich data typing:** XSD supports many built-in data types (e.g., `xs:integer`, `xs:date`, `xs:decimal`, `xs:boolean`) and allows defining custom types with precise constraints (e.g., a string matching a specific pattern, or a number within a specific range) — DTDs only support basic character data with no real type-checking.
- 3. Namespace support:** XSD is fully namespace-aware, integrating cleanly with the XML Namespace mechanism described above, whereas DTDs have very limited, awkward namespace support.
- 4. More expressive structural constraints:** XSD supports more sophisticated content models (e.g., specifying minimum/maximum occurrence counts precisely, choices between alternative structures, and extending/restricting existing type definitions through inheritance-like mechanisms), offering much finer control than DTD’s simpler grammar notation.

For these reasons, XSD has largely superseded DTD as the preferred schema/validation mechanism for modern XML-based applications and data exchange formats (e.g., SOAP web services, many industry-standard data formats), even though DTDs remain simple and adequate for smaller, less demanding use cases.

Q4. Explain XPath with example expressions, and describe how it is used as the foundation for XSLT.

XPath provides a concise syntax for addressing/selecting specific parts of an XML document's tree structure, treating the document as a hierarchy of nodes (elements, attributes, text) navigable via path-like expressions, conceptually similar to navigating directories in a file system.

Using the bookstore XML example from Q1, sample XPath expressions: - /bookstore — selects the root bookstore element. - /bookstore/book — selects **all** book child elements directly under bookstore. - /bookstore/book[1] — selects the **first** book element (XPath indexing starts at 1, not 0). - /bookstore/book[@category='fiction'] — selects all book elements whose category attribute equals "fiction" (using @ to reference an attribute). - //title — selects **all** title elements anywhere in the document, regardless of their position in the hierarchy (the // denotes a recursive/descendant search). - /bookstore/book[price>12]/title — selects the title of any book whose price child element's value exceeds 12 (demonstrating XPath's support for simple comparison predicates).

XPath as the foundation for XSLT: XSLT is a language used to transform an XML document into another output format (commonly HTML, for display, or a different XML structure). An XSLT stylesheet consists of a series of **template rules**, each specifying an XPath expression (in a match attribute) identifying which node(s) the rule applies to, and the corresponding output to generate when a matching node is processed:

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/bookstore">
    <html><body>
      <h2>My Book Collection</h2>
      <table border="1">
        <tr><th>Title</th><th>Author</th><th>Price</th></tr>
        <xsl:for-each select="book">
          <tr>
            <td><xsl:value-of select="title"/></td>
            <td><xsl:value-of select="author"/></td>
            <td><xsl:value-of select="price"/></td>
          </tr>
        </xsl:for-each>
      </table>
    </body></html>
  </xsl:template>
</xsl:stylesheet>
```

Here, <xsl:for-each select="book"> uses an XPath expression to iterate over every book element, and <xsl:value-of select="title"/> uses XPath to extract and output the text content of each book's title child element — this demonstrates that **XPath is the underlying addressing mechanism that XSLT relies on** to select which parts of the source XML to process and extract at each step of the transformation, making a solid understanding of XPath essential before working effectively with XSLT.

Q5. Explain the role of XML Processors (Parsers), comparing DOM-based and SAX-based parsing approaches with their respective advantages and use cases.

An **XML Processor (Parser)** is the software component responsible for reading raw XML text, checking it for correct syntax (well-formedness, and optionally validity against a DTD/Schema), and exposing its structured content to an application program through a defined API. Two fundamentally different parsing models are widely used:

DOM (Document Object Model) Parsing: The parser reads the **entire** XML document and constructs a complete, in-memory **tree representation**, where every element, attribute, and text node becomes an object in this tree, fully navigable in any direction (up to parents, down to children, sideways to siblings) and modifiable (elements/attributes can be added, changed, or removed programmatically).

Advantages: Full random access to any part of the document at any time, in any order; supports both reading and **modifying/writing** XML content, since the entire structure exists in memory as manipulable objects. *Disadvantages:* Requires loading the **entire** document into memory as this object tree, which can be prohibitively memory-intensive for very large XML documents (e.g., hundreds of megabytes), and there is some upfront processing delay to fully parse and build the tree before any part of it can be used. *Use case:* Best suited for applications that need to navigate a document's structure repeatedly or non-sequentially, or that need to programmatically **modify** the XML content (e.g., an XML editor application, or a program that needs to add/remove specific elements before re-saving the document).

SAX (Simple API for XML) Parsing: The parser reads through the XML document **sequentially**, from beginning to end, and as it encounters each syntactic construct (the start of an element, the end of an element, character data, etc.), it triggers a corresponding **event/callback** to the application's registered event-handler functions, without ever building or retaining an in-memory tree of the whole document.

Advantages: Very memory-efficient, since only a small amount of state needs to be held at any time (not the entire document), making it well suited for very large XML documents; also generally faster for simple, single-pass processing tasks since there's no overhead of building a full tree structure. *Disadvantages:* Strictly **forward-only, read-only** access — the application cannot go back to re-examine earlier parts of the document (unless it explicitly stores information itself as events arrive) and cannot use SAX to modify/write XML content directly. *Use case:* Best suited for large XML documents where only a single sequential pass is needed to extract specific information (e.g., searching a very large XML log/data file for records matching certain criteria) or for streaming XML processing scenarios where memory efficiency and processing speed matter more than random access or the ability to modify the document.

Choosing between them: In practice, DOM is preferred for smaller documents requiring flexible navigation or in-place editing, while SAX (or similar streaming APIs) is preferred for large documents or performance/memory-critical applications requiring only a straightforward, single-pass extraction of specific data.

UNIT – III: JavaScript & Ajax

2 Marks Questions

Q1. What is JavaScript? JavaScript is a lightweight, interpreted, client-side scripting language that runs inside web browsers, used to add interactivity, dynamic content updates, form validation, and event-driven behavior to web pages, and can also run server-side (e.g., via Node.js).

Q2. How are variables declared in JavaScript? Name the three keywords. Variables in JavaScript are declared using `var` (function-scoped, older style), `let` (block-scoped, allows reassignment), or `const` (block-scoped, cannot be reassigned after initialization) — `let` and `const` were introduced in ES6 and are now generally preferred over `var`.

Q3. What is a Function in JavaScript? A function is a reusable block of code defined once and executed (called) as needed, optionally accepting parameters and returning a value, e.g.,
`function add(a, b) { return a + b; }.`

Q4. Name any four JavaScript operators and their categories. Arithmetic operators (+, -, *, /), Comparison operators (==, ===, <, >), Logical operators (&&, ||, !), and Assignment operators (=, +=, -=) are four categories of JavaScript operators.

Q5. What is the DOM (Document Object Model) in JavaScript? The DOM is a programming interface/tree representation of an HTML (or XML) document, where every element, attribute, and piece of text is represented as a node object that JavaScript can access, traverse, and manipulate dynamically (e.g., `document.getElementById("myDiv")`).

Q6. What is an Event and an Event Handler in JavaScript? An event is an action or occurrence detected by the browser (e.g., a mouse click, key press, page load). An event handler is a JavaScript function registered to run automatically in response to a specific event on a specific element (e.g., `button.addEventListener("click", myFunction)`).

Q7. What is a JavaScript Closure? A closure is a function that “remembers” and continues to have access to variables from its enclosing (outer) lexical scope, even after that outer function has finished executing, allowing data to be encapsulated/persisted privately across function calls.

Q8. What is Ajax? Ajax (Asynchronous JavaScript and XML) is a web development technique that allows a web page to send and receive data from a server **asynchronously**, in the background, without requiring a full page reload, enabling more responsive, dynamic user interfaces.

Q9. What was the primary Pre-Ajax technique used for client-server communication? Before Ajax, techniques like using hidden `<iframe>` elements to load content in the background, or performing full-page reloads/form submissions, were used to update page content from the server — both far less smooth and responsive than Ajax’s asynchronous approach.

Q10. What is the XMLHttpRequest object? The `XMLHttpRequest` object is the core browser API object that enables Ajax — it allows JavaScript to make an HTTP request to a server and

receive the response asynchronously (via callback/event handling), without blocking or reloading the page.

Q11. Name two data formats commonly used in Ajax communication. XML and JSON (JavaScript Object Notation) are two data formats commonly used to exchange data between client and server in Ajax applications; JSON is generally preferred today due to its lighter weight and native compatibility with JavaScript.

Q12. Name one security concern associated with Ajax applications. Cross-Site Scripting (XSS) is a major security concern with Ajax applications — since Ajax dynamically injects server-retained content into the page, unescaped/unsanitized data from an untrusted source can allow an attacker to inject and execute malicious scripts in the context of the victim's page.

6 Marks Questions

Q1. Explain JavaScript variables, functions, operators, conditional statements, and loops with examples.

Variables: JavaScript variables are declared using `var`, `let`, or `const`:

```
let name = "Alice";           // block-scoped, reassignable
const PI = 3.14159;           // block-scoped, constant
var count = 0;                 // function-scoped (older style)
```

Functions: Reusable blocks of code, defined using the function keyword or arrow syntax:

```
function square(x) {
    return x * x;
}
const cube = (x) => x * x * x; // ES6 arrow function
console.log(square(4)); // 16
```

Operators: JavaScript supports arithmetic (+ - * / % **), comparison (== === != !== < > <= >=), logical (&& || !), and assignment (= += -= *= /=) operators:

```
let a = 10, b = 3;
console.log(a + b); // 13 (arithmetic)
console.log(a === "10"); // false (strict equality checks type too)
console.log(a > 5 && b < 5); // true (logical AND)
```

Conditional Statements: `if...else` and `switch` statements control program flow based on conditions:

```
let age = 20;
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
}
```

Loops: for, while, and do...while loops repeat a block of code:

```
for (let i = 0; i < 5; i++) {  
  console.log("Iteration " + i);  
}  
let i = 0;  
while (i < 3) {  
  console.log(i);  
  i++;  
}
```

Together, these constructs form the fundamental building blocks of JavaScript programs, allowing dynamic computation, decision-making, and repetition directly within the browser, forming the foundation for more advanced client-side behaviors like DOM manipulation and Ajax.

Q2. Explain the DOM (Document Object Model), Events, and Event Handling in JavaScript with a practical example.

The **DOM** represents an HTML document as a tree of node objects, which JavaScript can access and manipulate to dynamically change page content, structure, or styling after the page has loaded, without requiring a server round-trip or page reload.

```
<p id="demo">Original text</p>  
<button id="btn">Click Me</button>  
  
<script>  
  // Selecting an element by its ID  
  const para = document.getElementById("demo");  
  const button = document.getElementById("btn");  
  
  // Registering an event handler for the 'click' event  
  button.addEventListener("click", function() {  
    para.textContent = "Text changed after click!";  
    para.style.color = "blue";  
  });  
</script>
```

Explanation: `document.getElementById("demo")` retrieves a reference to the `<p>` element as a DOM node object. `addEventListener("click", ...)` registers an **event handler** — a callback function that the browser will automatically invoke whenever the user clicks the button. Inside the handler, `para.textContent` and `para.style.color` directly modify the live DOM tree, and the browser automatically **re-renders** the affected part of the page to reflect these changes — no page reload is needed. This DOM manipulation capability, combined with event handling, is the foundation of all interactive, dynamic client-side behavior in modern web applications, and is also what Ajax callbacks typically use to update the page once new data arrives asynchronously from the server.

Q3. Explain JavaScript Closures with an example, and describe their practical use cases.

A **closure** is created when a function is defined inside another function and “remembers” (retains access to) the variables of its enclosing scope, even after the outer function has already finished executing and would normally have its local variables discarded.

```
function createCounter() {  
    let count = 0; // private variable, enclosed by the closure  
  
    return function() {  
        count++; // inner function still has access to 'count'  
        return count;  
    };  
}  
  
const counter1 = createCounter();  
console.log(counter1()); // 1  
console.log(counter1()); // 2  
console.log(counter1()); // 3  
  
const counter2 = createCounter(); // a completely independent closure  
console.log(counter2()); // 1 (starts fresh, separate from counter1)
```

Explanation: Each call to `createCounter()` creates a **new, independent** count variable, and the returned inner function forms a closure over that specific count. Even though `createCounter()` has already returned and its execution context has technically ended, the inner function retains a live reference to its count variable, allowing it to persist and be updated across multiple calls to `counter1()`. Crucially, count is **not** directly accessible from outside — there’s no way to read or modify it except by calling the returned inner function — providing a form of **data encapsulation/privacy**, similar to a private variable in object-oriented programming.

Practical use cases: Closures are commonly used for (1) creating private state/variables (as in the counter example above, avoiding global variable pollution), (2) implementing **callback functions** that need to remember context/parameters from when they were created (very common in Ajax code, where a callback needs to remember details about the specific request it belongs to), and (3) the **module pattern**, where closures are used to expose only a specific public interface while keeping implementation details private within an enclosing function scope.

Q4. Explain the concept of Ajax, Pre-Ajax communication techniques, and the working of the XMLHttpRequest object with example code.

Ajax (Asynchronous JavaScript and XML) is a technique that allows a web page to communicate with a server **in the background**, sending and receiving data without requiring the browser to reload the entire page — enabling smooth, app-like interactivity (e.g., autocomplete search suggestions, live form validation against a server, infinite-scroll content loading).

Pre-Ajax Techniques: Before the widespread adoption of the XMLHttpRequest object, developers used more limited/awkward techniques to achieve similar (though far less smooth)

effects: - **Full page reloads / form submissions:** Simply submitting a form and letting the browser reload the entire page with the server's response — functional, but causes a visible flash/reload and discards all client-side state. - **Hidden <iframe> techniques:** Loading content into an invisible, hidden <iframe> element in the background, and then using JavaScript to read/extract the content from that iframe once loaded — a clever workaround, but cumbersome, with cross-domain restrictions and awkward error handling.

The XMLHttpRequest (XHR) Object: This is the core browser API that made true Ajax possible, allowing JavaScript to issue an HTTP request and handle the response **asynchronously**, via callback functions, without blocking the rest of the page's execution or requiring a reload.

```
const xhr = new XMLHttpRequest();

xhr.onreadystatechange = function() {
  if (xhr.readyState === 4 && xhr.status === 200) {
    // Request completed successfully
    document.getElementById("result").innerHTML = xhr.responseText;
  }
};

xhr.open("GET", "/api/getData?id=5", true); // true = asynchronous
xhr.send();
```

Explanation of the working: `new XMLHttpRequest()` creates a new request object. `xhr.open(method, url, async)` configures the request (here, an asynchronous GET request to a specific URL) without actually sending it yet. `xhr.onreadystatechange` registers a callback function that the browser will invoke automatically whenever the request's `readyState` changes (progressing through stages like “opened,” “headers received,” “loading,” and finally “done”). Inside the callback, checking `xhr.readyState === 4` confirms the request has fully completed, and `xhr.status === 200` confirms the server responded successfully (HTTP OK); only then is it safe to use `xhr.responseText` (the server's returned data) to update the page — here, injecting it into an element with id “result”. Finally, `xhr.send()` actually dispatches the request to the server, and the rest of the page's JavaScript continues executing immediately, without waiting/blocking — this asynchronous, non-blocking behavior is the defining characteristic that makes Ajax so much smoother than the pre-Ajax techniques described above. (Modern code more commonly uses the newer, promise-based `fetch()` API instead of raw `XMLHttpRequest`, but XHR remains foundational to understanding how Ajax works under the hood.)

Q5. Explain the Data Formats used in Ajax (XML vs JSON), Security Concerns in Ajax applications, and User Interface Design considerations for Ajax-based applications.

Data Formats — XML vs. JSON: Ajax responses need to be structured so JavaScript can easily parse and use them. Two common formats are: - **XML:** Structured, self-descriptive, hierarchical, and can be validated against a DTD/Schema, but is comparatively verbose and requires explicit parsing (e.g., using `responseXML` and DOM traversal) to extract data in JavaScript. - **JSON (JavaScript Object Notation):** A lightweight, text-based format that maps directly onto native JavaScript object/array syntax, e.g., `{ "name": "Alice", "age": 25, "hobbies":`

["reading", "coding"] } }. JSON is generally **preferred** in modern Ajax applications because it is more compact than XML, and can be parsed into a usable JavaScript object with a single call (`JSON.parse(responseText)`), rather than requiring more verbose DOM-traversal code as with XML.

Security Concerns in Ajax:

- 1. Cross-Site Scripting (XSS):** Since Ajax dynamically injects server-retained content directly into the page's DOM (e.g., via `innerHTML`), failing to properly escape/sanitize untrusted data before insertion can allow an attacker to inject malicious `<script>` content that executes in the context of the victim's session, potentially stealing cookies/session tokens or performing actions on the user's behalf.
- 2. Cross-Site Request Forgery (CSRF):** Since Ajax requests can be triggered automatically by page scripts (potentially from a malicious third-party page, if the victim is still logged into the target site), servers must validate that state-changing Ajax requests genuinely originated from the legitimate application (e.g., using anti-CSRF tokens), not from a forged cross-site request riding on the victim's existing authenticated session/cookies.
- 3. Exposure of sensitive logic/data:** Since Ajax calls are visible in the browser's network tools, sensitive business logic or unvalidated trust in client-side checks (e.g., performing authorization checks only in JavaScript, without re-verifying on the server) can be inspected or bypassed by a motivated attacker — all security-critical validation must always be enforced server-side as well, regardless of any client-side Ajax-based checks.

User Interface Design for Ajax: Since Ajax requests happen asynchronously and may take a noticeable amount of time (especially on slow connections), good UI design should: (1) provide clear **visual feedback** during a pending request (e.g., a loading spinner or "Saving..." indicator), so users understand the application hasn't frozen; (2) handle **errors gracefully** (e.g., displaying a clear, user-friendly error message if the Ajax request fails or times out, rather than leaving the interface in a broken or unresponsive state); and (3) preserve basic web usability expectations (e.g., ensuring browser back/forward navigation and bookmarking still work sensibly, which can be broken if an application relies too heavily on Ajax-driven content changes without properly updating the URL/history state, e.g., via the History API).

UNIT – IV: Java Servlets, MERN & LAMP Stack

2 Marks Questions

Q1. What is a Java Servlet? A Java Servlet is a server-side Java program that runs within a web/application server (e.g., Apache Tomcat), extending the server's capabilities by dynamically generating web content (typically HTML) in response to client HTTP requests, following the Java Servlet API specification.

Q2. What is CGI (Common Gateway Interface), and how do Servlets improve upon it? CGI is an older technology for generating dynamic web content, where each request spawns a **new separate process** to run an external script, which is resource-intensive and slow under load. Servlets improve on this by running as lightweight **threads within a single, persistent**

server process/JVM, avoiding the overhead of repeatedly starting new processes for each request.

Q3. List any three benefits of Java Servlets. (1) **Performance:** Servlets use lightweight threads (not new OS processes per request) within a persistent JVM, making them much faster than CGI under load; (2) **Platform independence:** Being written in Java, servlets can run on any server platform supporting a compatible JVM; (3) **Robustness:** Servlets benefit from Java's strong type-checking, exception handling, and automatic memory management (garbage collection), reducing common programming errors.

Q4. Name the main lifecycle methods of a Java Servlet. `init()` (called once, when the servlet is first loaded, for one-time setup), `service()` (called for every incoming request, typically dispatching to `doGet()` / `doPost()` based on the HTTP method), and `destroy()` (called once, when the servlet is being unloaded/shut down, for cleanup).

Q5. What is the difference between `doGet()` and `doPost()` in a Servlet? `doGet()` handles incoming HTTP GET requests (typically for retrieving data, often via query-string parameters), while `doPost()` handles incoming HTTP POST requests (typically for submitting/changing data, with parameters sent in the request body) — both are methods a servlet developer overrides to implement request-specific logic.

Q6. How does a Servlet read data submitted by a client? A servlet reads client-submitted data using the `HttpServletRequest` object's methods, such as `request.getParameter("name")` to retrieve a specific form/query parameter's value by name.

Q7. What is a Cookie in the context of web applications? A cookie is a small piece of data that a server sends to a client's browser (via an HTTP response header), which the browser then automatically stores and re-sends back to the same server with every subsequent request, commonly used to maintain state (e.g., login status, preferences) across otherwise stateless HTTP requests.

Q8. What is Session Tracking, and why is it needed? Session tracking is the technique of maintaining state/data associated with a specific user across multiple, separate HTTP requests, needed because HTTP itself is a **stateless** protocol (each request is independent, with no inherent memory of previous requests from the same client) — commonly implemented using cookies (storing a session ID) or URL rewriting.

Q9. What is the MERN stack? Expand the acronym. MERN stands for **M**ongoDB (NoSQL database), **E**xpress.js (Node.js web application framework), **R**ead (front-end JavaScript UI library), and **N**ode.js (JavaScript runtime for server-side execution) — a popular full-stack JavaScript web development stack using JavaScript across the entire application (client, server, and query logic).

Q10. What is the LAMP stack? Expand the acronym. LAMP stands for **L**inux (operating system), **A**pache (web server), **M**ySQL (relational database), and **P**HP (server-side scripting language) — a classic, widely used open-source web application stack.

Q11. What is React JS? React JS is a JavaScript library (developed by Facebook/Meta) for building interactive user interfaces using a component-based architecture, where the UI is

broken into reusable, composable components, and updates efficiently using a **Virtual DOM** to minimize actual browser DOM manipulation.

6 Marks Questions

Q1. Explain the Life Cycle of a Java Servlet with its key methods, and compare Servlets with CGI programming.

A Java Servlet's life cycle, managed by the **Servlet Container** (e.g., Apache Tomcat), consists of three key phases:

1. **Loading and Instantiation:** When the servlet is first requested (or at server startup, if pre-configured), the container loads the servlet class and creates a **single instance** of it.
2. **Initialization — `init()`:** The container calls the servlet's `init(ServletConfig config)` method **exactly once**, immediately after instantiation, before the servlet handles any requests. This is where one-time setup logic goes (e.g., opening a database connection pool, loading configuration).
3. **Request Handling — `service()`:** For **every** client request directed to this servlet, the container invokes the `service(ServletRequest, ServletResponse)` method. In the common `HttpServlet` subclass, this default implementation automatically examines the HTTP method of the request and dispatches to the corresponding method — `doGet()` for GET requests, `doPost()` for POST requests, etc. — which the developer overrides with request-specific logic. Critically, the **same single servlet instance** handles all incoming requests concurrently (each on its own thread), which is far more efficient than spawning a new process per request.
4. **Destruction — `destroy()`:** When the container decides to unload the servlet (e.g., during server shutdown, or application redeployment), it calls `destroy()` exactly once, allowing the servlet to release any held resources (e.g., closing database connections) before being garbage collected.

```
public class HelloServlet extends HttpServlet {
    public void init() {
        System.out.println("Servlet initialized");
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse
resp)
        throws ServletException, IOException {
        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<h1>Hello, " + req.getParameter("name") +
"!</h1>");
    }
    public void destroy() {
        System.out.println("Servlet destroyed");
    }
}
```

```
}  
}
```

Comparison with CGI: In the older CGI model, the web server spawns a **brand-new operating system process** for every single incoming request, running an external script/executable that generates the response and then terminates — this process creation/termination overhead is significant, and CGI provides no built-in mechanism to share resources (e.g., a database connection pool) across requests. Servlets, by contrast, run as **lightweight threads within a single, long-lived JVM process** managed by the servlet container — the servlet instance itself is created only once (via `init()`) and reused across all subsequent requests, avoiding repeated process-startup overhead, and it can maintain shared, in-memory state/resources (e.g., a connection pool, or a cache) across requests far more efficiently than CGI's process-per-request model allows. This is the primary reason servlets (and similar in-process web application technologies) largely replaced CGI for high-traffic dynamic web applications.

Q2. Explain how a Servlet reads client data, and describe HTTP Request and Response Headers with examples.

Reading Client Data: A servlet accesses data submitted by the client (whether via URL query parameters on a GET request, or form-body parameters on a POST request) through the `HttpServletRequest` object passed into its `doGet()`/`doPost()` method:

```
protected void doPost(HttpServletRequest request, HttpServletResponse  
response)  
    throws ServletException, IOException {  
    String username = request.getParameter("username");    // single  
value  
    String[] hobbies = request.getParameterValues("hobby"); //  
multiple values (checkboxes)  
    String userAgent = request.getHeader("User-Agent");    // reading  
a request header  
  
    response.setContentType("text/html");  
    PrintWriter out = response.getWriter();  
    out.println("<p>Welcome, " + username + "!</p>");  
}
```

`request.getParameter(name)` retrieves the value of a single named parameter (whether it came from the query string or the request body, depending on the HTTP method used), and `request.getParameterValues(name)` retrieves an array of all values for a parameter that can have multiple selections (e.g., a group of checkboxes sharing the same name).

HTTP Request Headers: Sent by the client with every request, providing metadata about the request — e.g., `Host` (target domain), `User-Agent` (browser/client identification string), `Accept` (content types the client can handle), `Cookie` (any cookies previously set for this domain), and `Content-Type/Content-Length` (for POST requests with a body). A servlet reads these via `request.getHeader("HeaderName")`.

HTTP Response Headers: Set by the servlet (via the `HttpServletResponse` object) before sending the response body, providing metadata about the response — e.g., `Content-Type` (telling the browser how to interpret the body, such as `text/html` or `application/json`), `Set-Cookie` (instructing the browser to store a cookie), `Location` (used with redirect responses), and `Cache-Control` (caching instructions). A servlet sets these via methods like `response.setContentType(...)` or `response.setHeader("HeaderName", "value")`.

Q3. Explain Cookies and Session Tracking in Java Servlets, with code examples.

Since **HTTP is stateless** (the server, by default, has no memory of previous requests from the same client), web applications need explicit mechanisms to maintain state (e.g., “is this user logged in?”) across multiple requests — this is achieved primarily through **Cookies** and **Session Tracking**.

Cookies: A servlet can create a cookie and attach it to its response; the browser then automatically stores this cookie and re-sends it with every subsequent request to the same domain:

```
// Setting a cookie
Cookie userCookie = new Cookie("username", "alice123");
userCookie.setMaxAge(60 * 60 * 24); // expires in 1 day (in seconds)
response.addCookie(userCookie);

// Reading cookies on a later request
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("username")) {
            String username = c.getValue();
        }
    }
}
```

Session Tracking (using `HttpSession`): Rather than manually managing individual cookies for every piece of state, the Servlet API provides a higher-level `HttpSession` object, which the container automatically associates with a specific client (typically by transparently using a single cookie behind the scenes, storing just a unique session ID, while the actual session **data** is kept safely on the **server**, not exposed to the client):

```
// Storing data in the session
HttpSession session = request.getSession(); // creates a new session
if none exists yet
session.setAttribute("username", "alice123");
session.setAttribute("cartItems", new ArrayList<String>());

// Retrieving data from the session on a later request
HttpSession session = request.getSession();
String username = (String) session.getAttribute("username");
```


Key distinction: Cookies store data **directly in the client's browser** (limited in size, and visible/editable by the client, so should not hold sensitive data directly), while `HttpSession` stores the actual data **server-side**, using only a small session-identifier cookie on the client to look up the corresponding server-side data on each request — making session tracking generally the safer, more scalable, and more flexible approach for maintaining substantial or sensitive per-user state (e.g., shopping cart contents, authentication status) in real web applications.

Q4. Explain the MERN stack and the LAMP stack, comparing their components and typical use cases.

LAMP Stack: A long-established, widely used open-source web application stack: - **L — Linux:** The operating system on which the entire stack runs. - **A — Apache (web server):** Handles incoming HTTP requests, serving static files directly and forwarding dynamic requests to the PHP interpreter. - **M — MySQL:** A relational (SQL) database used to store and query the application's structured data. - **P — PHP:** A server-side scripting language (covered in depth in Unit V) that generates dynamic HTML content, typically by querying the MySQL database and embedding results into HTML templates.

LAMP has powered a huge portion of the traditional web (including platforms like WordPress) for decades, valued for its maturity, huge ecosystem/community support, and relatively simple, well-understood deployment model for traditional server-rendered web applications.

MERN Stack: A more modern, all-JavaScript full-stack combination, popular for building single-page applications (SPAs) with rich, dynamic client-side interactivity: - **M — MongoDB:** A NoSQL, document-oriented database, storing data as flexible, JSON-like documents rather than rigid relational tables. - **E — Express.js:** A minimal, flexible web application framework running on Node.js, used to build the server-side API (typically a RESTful JSON API) that the front-end communicates with. - **R — React:** A component-based JavaScript library for building the client-side user interface, using a Virtual DOM for efficient, dynamic UI updates without full page reloads. - **N — Node.js:** The JavaScript runtime environment that executes the server-side (Express) code, allowing the **same language (JavaScript)** to be used across the entire stack — client-side (React), server-side (Express/Node), and even the data format (JSON, which maps naturally both to MongoDB documents and JavaScript objects).

Comparison and Use Cases: LAMP follows a more traditional **server-rendered** model (the server typically generates complete HTML pages, though it can also serve APIs), well suited to more traditional websites, content management systems, and applications where full-page loads/simpler architecture are acceptable. MERN follows a more modern **API-driven, single-page application (SPA)** model, where the server (Express/Node) exposes a JSON API, and the client-side React application handles all UI rendering and dynamically fetches/updates data via Ajax-style calls to that API — well suited to highly interactive, app-like web experiences (e.g., dashboards, social media feeds, real-time collaborative tools) where minimizing full-page reloads and providing a smooth, responsive UI is a priority. The choice between such stacks in practice often depends on team expertise (all-JavaScript vs. a PHP/SQL background), the specific data model needs (structured relational data suits MySQL; flexible/nested/rapidly evolving data suits MongoDB), and the desired user-experience style (traditional multi-page vs. app-like SPA).

Q5. Explain the basics of React JS, including components, JSX, and the Virtual DOM concept.

React JS is a JavaScript library for building user interfaces through a **component-based** architecture — a complex UI is broken down into smaller, independent, reusable **components**, each responsible for rendering a piece of the overall interface and managing its own internal state/logic.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0); // component's local state

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

export default Counter;
```

JSX: React components are typically written using **JSX** (JavaScript XML), a syntax extension that allows writing HTML-like markup directly within JavaScript code (as seen in the `return ( ... )` statement above), which is then compiled (via a tool like Babel) into regular JavaScript function calls (`React.createElement( ... )`) before being executed by the browser. JSX makes component templates much more readable and intuitive to write compared to constructing the same UI structure using plain JavaScript DOM API calls.

State and Re-rendering: The `useState` hook (shown above) gives a component its own piece of internal, reactive state (`count`), along with a setter function (`setCount`) to update it. Whenever `setCount` is called, React automatically re-renders the component (and any of its children that depend on the changed state) to reflect the new value — the developer simply describes **what** the UI should look like for a given state, rather than manually writing imperative code to update specific DOM elements when data changes.

The Virtual DOM: Directly manipulating the real browser DOM is often a significant performance bottleneck for complex, frequently updating UIs. React addresses this using a **Virtual DOM** — a lightweight, in-memory JavaScript representation of the UI tree. Whenever a component's state changes, React first re-renders the component into a new Virtual DOM tree, then efficiently **compares (diffs)** this new tree against the previous one to determine the **minimal set of actual changes** needed, and only then applies exactly those specific, minimal updates to the real browser DOM. This diffing/reconciliation process allows React applications to update the UI efficiently even with frequent state changes, without the performance cost of naively re-rendering (and touching) the entire real DOM on every single change — a key reason for React's popularity in building complex, highly interactive, and performant user

interfaces, and a natural client-side counterpart to the server-side API architecture (as in the MERN stack) discussed above.

UNIT – V: PHP & Database Access

2 Marks Questions

Q1. What is PHP? PHP (Hypertext Preprocessor) is a widely used, open-source, server-side scripting language designed specifically for web development, embedded directly within HTML pages and executed on the server to generate dynamic content before the resulting HTML is sent to the client's browser.

Q2. What are the basic syntactic delimiters used to embed PHP code within HTML? PHP code is embedded within `<?php ... ?>` tags inside an HTML file; the web server executes any code within these tags and outputs the result, while content outside these tags is passed through to the browser unchanged, as plain HTML.

Q3. Name the primitive data types supported in PHP. PHP supports the primitive types: **Integer, Float (Double), String, Boolean**, along with special types **Array, Object, and NULL**.

Q4. How are variables declared in PHP? PHP variables are declared simply by prefixing an identifier with a dollar sign (\$), without needing an explicit type declaration (PHP is dynamically typed), e.g., `$name = "John";` or `$age = 25;`.

Q5. Name any two control statements available in PHP. `if...else` (conditional branching) and `for/while/foreach` (loops) are common control statements in PHP, syntactically very similar to their counterparts in C/Java/JavaScript.

Q6. What is an Associative Array in PHP? An associative array in PHP is an array that uses named keys (strings) instead of purely sequential numeric indices to access its elements, e.g., `$person = array("name" => "Alice", "age" => 25);`, functioning similarly to a dictionary/map in other languages.

Q7. How does PHP support pattern matching? PHP supports pattern matching primarily through **regular expressions**, using functions such as `preg_match()` (to test if a pattern matches, and extract matched groups) and `preg_replace()` (to find and replace text matching a pattern), based on PCRE (Perl-Compatible Regular Expressions) syntax.

Q8. How does a PHP script access data submitted via an HTML form? PHP provides superglobal arrays `$_GET` and `$_POST` (and `$_REQUEST`, which merges both) to access form data submitted via GET or POST methods, respectively, e.g., `$username = $_POST['username'];`.

Q9. How does PHP handle file operations? Name one relevant function. PHP provides built-in functions for file handling, such as `fopen()` (open a file), `fwrite()` (write data to a file), `fread()` (read data from a file), and `fclose()` (close the file handle) — enabling scripts to read/write files on the server's filesystem.

Q10. How does PHP set and read Cookies? PHP sets a cookie using the `setcookie(name, value, expiry)` function (which must be called before any HTML output), and reads previously set cookies via the superglobal `$_COOKIE` array, e.g., `$_COOKIE['username']`.

Q11. How does PHP implement Session Tracking? PHP implements session tracking by calling `session_start()` at the beginning of a script (which either creates a new session or resumes an existing one, typically identified via a session-ID cookie), and then storing/retrieving session data using the superglobal `$_SESSION` array, e.g., `$_SESSION['username'] = "alice";`.

Q12. What is JDBC? JDBC (Java Database Connectivity) is a Java API that provides a standard, vendor-independent way for Java applications (including servlets) to connect to, query, and update relational databases, regardless of the specific underlying database vendor (MySQL, Oracle, PostgreSQL, etc.), as long as an appropriate JDBC driver is available.

6 Marks Questions

Q1. Explain the general syntactic characteristics of PHP, including primitives, expressions, and control statements, with an example script.

PHP scripts are embedded directly within HTML files using `<?php ... ?>` delimiters; anything outside these tags is sent to the browser as-is (plain HTML), while code inside is executed by the PHP interpreter on the server before the page is sent.

```
<!DOCTYPE html>
<html>
<body>

<h2>PHP Basics Demo</h2>

<?php
    // Primitives and variable declaration (no explicit type needed)
    $name = "Alice";           // string
    $age = 25;                 // integer
    $gpa = 8.75;              // float
    $isStudent = true;        // boolean

    // Expressions and operators
    $nextYearAge = $age + 1;

    // Control statement: if...else
    if ($age >= 18) {
        echo "<p>$name is an adult.</p>";
    } else {
        echo "<p>$name is a minor.</p>";
    }

    // Loop: for
```

```

echo "<p>Countdown: ";
for ($i = 5; $i > 0; $i--) {
    echo $i . " ";
}
echo "</p>";

// Array and foreach loop
$fruits = array("apple", "banana", "cherry");
echo "<ul>";
foreach ($fruits as $fruit) {
    echo "<li>$fruit</li>";
}
echo "</ul>";
?>

</body>
</html>

```

Explanation: PHP variables always begin with \$ and require no type declaration (PHP determines the type dynamically at runtime based on the assigned value). String interpolation is supported directly within double-quoted strings (e.g., "\$name is an adult" automatically substitutes the variable's value). PHP's control statements (if/else, for, while, foreach) use syntax very similar to C/Java/JavaScript, making it relatively easy to learn for programmers already familiar with those languages. The echo statement is PHP's primary mechanism for outputting content (typically HTML) to the response sent back to the browser, and PHP code can be freely interspersed with static HTML markup, allowing dynamic content generation to be woven directly into an otherwise normal HTML page structure.

Q2. Explain Functions and Pattern Matching (Regular Expressions) in PHP with examples.

Functions in PHP: Defined using the function keyword, PHP functions can accept parameters (optionally with default values) and return a value:

```

<?php
function calculateDiscount($price, $discountPercent = 10) {
    $discountAmount = $price * ($discountPercent / 100);
    return $price - $discountAmount;
}

$finalPrice = calculateDiscount(1000); // uses default 10%
$finalPriceCustom = calculateDiscount(1000, 25); // uses 25%
echo "Final price (default): $finalPrice<br>";
echo "Final price (custom): $finalPriceCustom<br>";
?>

```

Pattern Matching (Regular Expressions): PHP supports powerful regular-expression-based pattern matching through PCRE (Perl-Compatible Regular Expressions) functions, most commonly preg_match() (test for a match, and optionally capture matched groups) and preg_replace() (search-and-replace using a pattern):

```

<?php
$email = "student@osmania.edu";

// Validate an email format using a regular expression
$pattern = "/^[\\w.+-]+@[\\w-]+\\. [a-zA-Z]{2,}$/";
if (preg_match($pattern, $email)) {
    echo "Valid email address.<br>";
} else {
    echo "Invalid email address.<br>";
}

// Extracting parts of a string using a pattern with capture groups
$date = "2026-09-01";
if (preg_match("/(\\d{4})-(\\d{2})-(\\d{2})/", $date, $matches)) {
    echo "Year: " . $matches[1] . ", Month: " . $matches[2] . ", Day: " . $matches[3];
}

// Replacing text matching a pattern
$text = "Contact us at 123-456-7890 or 987-654-3210";
$cleaned = preg_replace("/\\d{3}-\\d{3}-\\d{4}/", "[phone number removed]", $text);
echo $cleaned;
?>

```

Explanation: `preg_match($pattern, $subject, $matches)` returns 1 if the pattern matches somewhere in the subject string (and 0 if not), and — if a `$matches` variable is provided — populates it with the full match (`$matches[0]`) plus any parenthesized capture groups (`$matches[1]`, `$matches[2]`, etc.). `preg_replace($pattern, $replacement, $subject)` returns a new string with every occurrence matching the pattern replaced by the specified replacement text. These pattern-matching capabilities are extensively used in PHP web applications for tasks such as validating user input formats (emails, phone numbers, postal codes), extracting specific structured data from larger text, and sanitizing/transforming text before storing or displaying it.

Q3. Explain PHP Form Handling and Session Tracking with example code.

Form Handling: When an HTML form submits data to a PHP script, PHP automatically populates the superglobal arrays `$_GET` (for GET-method form submissions) or `$_POST` (for POST-method submissions) with the submitted field values, keyed by each input's name attribute:

```

<form action="process.php" method="POST">
    Name: <input type="text" name="username"><br>
    Email: <input type="email" name="email"><br>
    <input type="submit" value="Submit">
</form>

<?php
// process.php

```

```

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = htmlspecialchars($_POST['username']); // sanitize to
prevent XSS
    $email = htmlspecialchars($_POST['email']);

    if (empty($username) || empty($email)) {
        echo "Please fill in all fields.";
    } else {
        echo "Thank you, $username! We will contact you at $email.";
    }
}
?>

```

Explanation: `$_SERVER["REQUEST_METHOD"]` lets the script confirm the request type before processing. `htmlspecialchars()` is a critical security function that escapes special HTML characters (like `<`, `>`, `&`) in user-submitted data before it is echoed back into the page, preventing **Cross-Site Scripting (XSS)** attacks where a malicious user might otherwise submit form data containing executable script tags.

Session Tracking: PHP provides built-in session support to persist data for a specific user across multiple page requests:

```

<?php
// login.php
session_start(); // must be called before any HTML output
$_SESSION['username'] = "alice123";
$_SESSION['loggedIn'] = true;
echo "Login successful!";
?>

<?php
// dashboard.php (a later request from the same user)
session_start(); // resumes the existing session, using the browser's
session-ID cookie
if (isset($_SESSION['loggedIn']) && $_SESSION['loggedIn'] === true) {
    echo "Welcome back, " . $_SESSION['username'] . "!";
} else {
    echo "Please log in first.";
    // typically: header("Location: login.php"); exit();
}
?>

```

Explanation: `session_start()` must be called at the very beginning of every script that needs to access session data (before any actual HTML output is sent), since it may need to send a session-ID cookie header to the browser. Once started, the `$_SESSION` superglobal array behaves like a persistent, server-side storage dictionary specific to that one user's browser session, allowing data (like login status) set in one script (`login.php`) to be read back in a completely separate, later request to a different script (`dashboard.php`) by the same user — this is PHP's direct equivalent to the `HttpSession` mechanism discussed for Java Servlets in Unit IV.

Q4. Explain the different architectures for Database Access from the Web, and describe accessing a database using PHP.

Architectures for Database Access: Web applications typically access a backend database using one of a few general architectural patterns: 1. **Direct server-side scripting + database driver:** The web/application server's scripting language (PHP, or a Java Servlet) directly uses a language-specific database extension/driver (e.g., PHP's `mysqli` or PDO extensions, or Java's JDBC) to connect to and query the database, embedding results directly into the dynamically generated response — this is the classic, simplest architecture (as used by the LAMP stack). 2. **API/middleware layer:** A dedicated backend API server (e.g., a Node.js/Express server in the MERN stack) handles all database access, exposing a well-defined (often RESTful, JSON-based) API that client-side applications (e.g., a React front-end) call via Ajax, decoupling the database access logic from the specific presentation/UI layer. 3. **ORM (Object-Relational Mapping) layers:** Rather than writing raw SQL queries directly, many modern frameworks use an ORM library that maps database tables/rows to programming-language objects, letting developers interact with the database using familiar object-oriented syntax rather than raw SQL strings, which the ORM translates into SQL behind the scenes.

Database Access with PHP (using `mysqli`):

```
<?php
// Connect to a MySQL database
$conn = new mysqli("localhost", "db_user", "db_password",
"student_db");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Execute a parameterized (prepared) query to prevent SQL injection
$stmt = $conn->prepare("SELECT name, marks FROM students WHERE branch
= ?");
$branch = "AI/ML";
$stmt->bind_param("s", $branch);
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo "Name: " . $row['name'] . ", Marks: " . $row['marks'] .
"<br>";
}

$stmt->close();
$conn->close();
?>
```

Explanation: `new mysqli(...)` establishes a connection to the MySQL server, specifying host, credentials, and target database. Using a **prepared statement** (`$conn->prepare(...)`) with a `?` placeholder, followed by `bind_param(...)` rather than directly concatenating user

input into a raw SQL string is a critical security practice, since it prevents **SQL Injection** attacks — the database driver safely handles the substitution of the actual parameter value, ensuring it is always treated strictly as data, never as executable SQL syntax, regardless of what characters the value contains. `fetch_assoc()` retrieves each resulting row as an associative array (keyed by column name), making it straightforward to embed the retrieved data into dynamically generated HTML output.

Q5. Explain Database Access with MongoDB and with JDBC, contrasting the NoSQL and relational (SQL) approaches to web database access.

Database Access with MongoDB (NoSQL, typically via Node.js in the MERN stack):

```
const { MongoClient } = require('mongodb');

async function getStudents() {
  const client = new MongoClient("mongodb://localhost:27017");
  await client.connect();
  const db = client.db("studentDB");
  const collection = db.collection("students");

  // Query documents matching a filter
  const students = await collection.find({ branch:
"AI/ML" }).toArray();
  console.log(students);

  await client.close();
}
```

MongoDB stores data as flexible, **JSON-like documents** (technically BSON) within **collections**, rather than rigid rows within tables governed by a fixed schema — a single collection can contain documents with varying structures/fields, which offers flexibility for rapidly evolving or naturally hierarchical/nested data (e.g., a student document that embeds an array of enrolled courses directly, rather than requiring a separate join table). Queries use a JSON-like filter syntax (e.g., `{ branch: "AI/ML" }`) rather than SQL.

Database Access with JDBC (relational, typically via Java Servlets):

```
import java.sql.*;

public class StudentDAO {
  public void printStudents() throws SQLException {
    String url = "jdbc:mysql://localhost:3306/student_db";
    Connection conn = DriverManager.getConnection(url, "db_user",
"db_password");

    PreparedStatement stmt = conn.prepareStatement(
      "SELECT name, marks FROM students WHERE branch = ?");
    stmt.setString(1, "AI/ML");
    ResultSet rs = stmt.executeQuery();
```

```

        while (rs.next()) {
            System.out.println("Name: " + rs.getString("name") +
                               ", Marks: " + rs.getInt("marks"));
        }
        conn.close();
    }
}

```

JDBC provides a standard, vendor-neutral Java API for connecting to and querying **relational (SQL)** databases; `DriverManager.getConnection(...)` establishes a connection (using a vendor-specific JDBC driver appropriate to the target database, e.g., MySQL, Oracle), and `PreparedStatement` (again, used here in its safe, parameterized form to prevent SQL injection, just as with PHP's `mysqli`) executes SQL queries against strictly-typed, schema-defined tables, returning results as a `ResultSet` that can be iterated row by row.

Contrast — NoSQL (MongoDB) vs. Relational (JDBC/SQL): Relational databases (accessed via JDBC, or PHP's `mysqli`/PDO) enforce a fixed, predefined schema (tables, columns, data types) and excel at representing structured data with well-defined relationships between entities (e.g., students, courses, enrollments), using SQL joins to combine related data cleanly, and providing strong consistency/ACID transactional guarantees. NoSQL databases like MongoDB offer a much more flexible, schema-less (or schema-optional) document model, better suited to rapidly evolving data structures, naturally nested/hierarchical data, or applications prioritizing horizontal scalability and development flexibility over strict relational integrity — this fundamental difference in data modeling philosophy is exactly why the choice between a MySQL/JDBC-based (LAMP-style) architecture and a MongoDB-based (MERN-style) architecture is one of the first major design decisions made when building a new web application's backend.

End of Question Bank — Prepared as per Osmania University B.E. AI/ML Web Programming (PC 601 AI) syllabus, with reference to Sebesta's "Programming the World Wide Web," Willard's "HTML5," and Powell's "Ajax: The Complete Reference."